AI/ML Engineer — Answer Sheet Evaluation

CNV AI Labs · Timed Technical Assignment

Duration: 120 minutes **Mode:** Live / proctored, remote **Tools:** Any AI tools allowed **Format:** Individual

The Problem

250 million students sit exams in India every year, graded by hand under real-world pressure — and the marks two evaluators give the same paper can differ by 15–20%. We're building AI that reads handwriting, understands context, scores semantically, and flags what it's unsure of. This assignment is a scaled-down, timed slice of that pipeline — enough to see how you think and build under a clock.

The Task

No sample data will be provided. Within the 120 minutes, you must first source your own sample answer sheet(s) — e.g. a scanned/photographed handwritten exam script, publicly available on the internet (exam board sample scripts, open datasets, GitHub/Kaggle answer-sheet datasets, or your own handwritten sample if nothing suitable is found quickly). Pick a subject and 3–5 questions you can reasonably define an answer key for. Then build a pipeline that extracts, structures, and scores the answers against your own answer key/rubric — and know when to flag its own uncertainty.

1. Extract

OCR / handwriting recognition to pull text from each answer sheet. Doesn't need to be perfect — show your approach to handling messy handwriting and mixed layouts (diagrams, margin notes, crossed-out lines).

2. Structure

Segment the extracted text by question number. Handle at least one case where an answer spills across pages or a diagram interrupts the text.

3. Score

For each answer, produce a score against the expected answer/rubric — based on meaning, not keyword overlap. A near-correct answer phrased differently should score close to a correct one.

4. Flag Confidence

For each scored answer, output a confidence level (e.g. high/medium/low) and a one-line reason. Low-confidence answers should be the ones you'd actually want a human to double-check.

Deliverables

Item	What we're looking for
Code / notebook	Working, runnable pipeline (script or notebook) covering all 4 steps above
Source sheet	Link/reference to where the sample answer sheet was sourced from, plus your own answer key/rubric for the questions attempted
Output file	A JSON or CSV with: question no., extracted answer text, score, score vs. your answer key, confidence label, reason
Approach note	10–15 lines: what you'd do differently with more time, and where the pipeline is weakest

Evaluation Criteria

Criterion	Weight
Extraction quality on real handwriting/layout	25%
Scoring logic quality & consistency vs. own answer key	30%
Confidence flagging — does it flag the right things	20%
Engineering judgment under time pressure (tradeoffs, what you skipped and why)	15%
Clarity of approach note	10%

Rules

- The 120 minutes includes time to find your sample answer sheet(s) — budget for this, don't spend the whole round searching.
- Hard stop at 120 minutes — submit whatever state the pipeline is in.
- Any AI tool is allowed (ChatGPT, Claude, Copilot, Cursor, etc.) — use them as you would on the job.
- Any language/framework is fine; Python is the most common choice for this stack.
- Pre-trained OCR/HTR models, APIs, and open-source libraries are all fair game — you are not expected to train anything from scratch in 2 hours.
- Screen share stays on throughout (live/proctored round).

What Happens Next

Shortlisted candidates move to a live oral defence with Sachin and Hemant — walking through every decision in this pipeline, with follow-up "what if" scenarios at scale (millions of pages, mixed scripts, exam-season load).

CNV AI Labs · HTR · Semantic Scoring · Confidence Calibration